

FE Interview Mastery — 5 Hour/Day Intensive Plan (10 Days)

Companion to FE_Interview_Prep_Plan.md and FE_Interview_Flashcards.md

Use **this doc for:** the revised daily schedule + all NEW questions not covered in the original plan.

REVISED 10-DAY SCHEDULE (5 hrs/day)

With 5 hours instead of 3, we compress 14 days → 10 days and add a 4th block for advanced/bonus topics + rapid mock practice.

Daily structure (300 min):

Block	Time	Focus
1	75 min	JS/TS fundamentals + ES6+
2	75 min	DSA (timed problem-solving)
3	75 min	React + Next.js
4	45 min	NEW: Advanced topics (this doc's Part 8)
5	30 min	Flashcard rapid-fire + explain-out-loud practice

Day	JS/TS	DSA	React/Next	Advance 8 below)
1	Execution context, closures, <code>this</code> , hoisting	Two Sum, Palindrome, Big-O	Virtual DOM, reconciliation, Fiber	Web Perf basics: C
2	Prototypes, event loop, <code>call/apply/bind</code>	Sliding window set	Hooks: <code>useState/useEffect</code> deep	Accessib deep dive
3	Promises, <code>async/await</code> , currying	Hashmap problems	<code>useMemo/useCallback/React.memo</code>	Testing: I patterns
4	ES6+ destructuring/spread/optional chaining	Recursion + flatten	Custom hooks, Context API	State ma Redux/Zi tradeoffs
5	Generators, Map/Set, modules	Two- pointer set	Controlled forms, code-splitting	Browser r pipeline & reflow/rej
6	Polyfills (<code>map/filter/Promise/debounce</code>)	Stack/queue problems	SSR/SSG/ISR/CSR	Security: CORS ba
7	Mock interview (JS+TS, 45 min)	Timed mixed set (45 min)	App Router, Server/Client Components	System d compone
8	TypeScript: interfaces/generics/unions	Practice set (timed)	Data fetching, caching, middleware	Build too Webpack bundling
9	TypeScript: utility types, narrowing, advanced	Practice set (timed)	Performance optimization patterns	System d an infinit
10	Full mock (JS+TS+React+Next, 90 min)	Full mock DSA (45 min)	Full mock (60 min)	Final flas all topics

Tip with the extra time: don't just read more — *build* things. Each day, rebuild one pattern from scratch without references (a hook, a polyfill, a small component) using block 4/5 time. Recall under pressure is what actually gets tested.

PART 8 — ADVANCED / BONUS QUESTIONS

State Management

Q1. Redux vs Context API vs Zustand — when would you choose each?

Context API is built-in and fine for low-frequency global state (theme, auth user) but re-renders all consumers on change. Redux gives predictable state updates via pure reducers, time-travel debugging, and middleware (great for large apps with complex, frequently-changing shared state), but adds boilerplate. Zustand offers Redux-like centralized state with far less boilerplate and no Provider wrapping requirement — often a good middle ground for medium apps.

Q2. What problem does Redux Toolkit solve compared to classic Redux?

Classic Redux required verbose action types, action creators, and immutable update logic written by hand. Redux Toolkit (RTK) bundles `createSlice` (auto-generates actions/reducers), uses Immer internally so you can "mutate" state in reducers safely, and includes RTK Query for data fetching/caching — drastically cutting boilerplate.

Q3. How do you decide what should live in local component state vs global state?

Local state: data only one component (and its children) needs, especially UI-only state (input value, toggle, modal open). Global state: data needed across unrelated parts of the tree, or that must persist across navigation (auth user, cart, theme). Rule of thumb: lift state only as high as necessary, and prefer local state by default.

Q4. What is prop drilling and what are the ways to avoid it?

Prop drilling is passing props through several intermediate components that don't use them, just to reach a deeply nested child. Solutions: Context API, component composition (passing `children` instead of drilling props), or a state management library.

Testing (RTL + Jest)

Q5. What's the philosophy behind React Testing Library (RTL), and how does it differ from Enzyme?

RTL encourages testing components the way a user interacts with them (querying by role/text/label, firing real events) rather than testing internal implementation details (state, instance methods) like Enzyme did. This makes tests more resilient to refactors — as long as behavior stays the same, tests stay green.

Q6. How do you test an async component that fetches data (e.g., your `useFetch` hook)?

Use `findBy*` queries (which wait for elements) or `waitFor`, combined with mocking the fetch call (`jest.mock` or `msw` for network-level mocking).

```
jsx

test('shows data after fetch', async () => {
  render(<UserProfile />);
  expect(screen.getByText(/loading/i)).toBeInTheDocument();
  const name = await screen.findByText('Sushil Sarraf');
  expect(name).toBeInTheDocument();
});
```

Q7. `getBy` vs `queryBy` vs `findBy` in RTL — when to use each?

`getBy*` throws immediately if not found (use when element should already be present).

`queryBy*` returns `null` instead of throwing (use to assert something is *absent*). `findBy*` returns a Promise and retries until found or timeout (use for async-appearing elements).

Q8. How do you mock a module or a specific function in Jest?

```
js

jest.mock('./api', () => ({
  fetchUser: jest.fn(() => Promise.resolve({ name: 'Sushil' })),
}));
```

For partial mocking, use `jest.spyOn(module, 'fn')` and restore afterward with `mockRestore()`.

Q9. How would you test that a button click triggers the right behavior?

```
jsx

test('calls onSave when clicked', () => {
  const onSave = jest.fn();
  render(<Button onClick={onSave} label="Save" />);
  fireEvent.click(screen.getByRole('button', { name: /save/i }));
  expect(onSave).toHaveBeenCalledTimes(1);
});
```

Q10. What is snapshot testing, and what's a common pitfall with it?

Snapshot testing renders a component and saves its output as a reference; future test runs

diff against it. Pitfall: snapshots can become "update and forget" — developers blindly accept diffs (`--ci=false`) / (`-u`) without actually reviewing them, defeating the purpose. Best used for small, stable UI pieces, not entire pages.

Web Performance & Core Web Vitals

Q11. What are the Core Web Vitals, and what does each measure?

- **LCP (Largest Contentful Paint)** — loading performance; time until the largest visible element renders. Target: under 2.5s.
- **INP (Interaction to Next Paint)** — responsiveness; delay between user interaction and visual response (replaced FID in 2024). Target: under 200ms.
- **CLS (Cumulative Layout Shift)** — visual stability; how much content unexpectedly shifts during load. Target: under 0.1.

Q12. How would you improve a poor LCP score?

Talking points: optimize/compress the largest image or hero element, use `next/image` or responsive images, preload critical resources, reduce server response time (TTFB), remove render-blocking CSS/JS, use a CDN.

Q13. What causes Cumulative Layout Shift, and how do you prevent it?

Causes: images/ads without reserved dimensions, web fonts causing FOUT/FOIT reflow, content injected above existing content dynamically. Prevention: always set width/height (or aspect-ratio) on media, use `next/font` to avoid font-swap layout shift, reserve space for dynamic content (skeleton loaders).

Q14. What's the difference between lazy loading and preloading, and when do you use each?

Lazy loading defers loading offscreen/non-critical resources until needed (images below the fold, route chunks) to speed initial load. Preloading tells the browser to fetch a critical resource early (a font, a hero image, the next likely route) so it's ready before needed. Use lazy loading for "not needed yet," preloading for "definitely needed very soon."

Q15. How does the browser's critical rendering path work?

HTML is parsed into the DOM, CSS into the CSSOM, the two combine into the **Render Tree**, the browser computes **Layout** (geometry/position), then **Paint** (pixels), then **Composite** (layering). CSS is render-blocking by default; JS can block HTML parsing unless marked `async` / `defer`.

Accessibility (ADA) Deep Dive

Q16. What are the four principles of accessible design (POUR)?

Perceivable (users can perceive the content via sight/sound/touch), Operable (interface is usable via keyboard, not just mouse), Understandable (content and UI behavior are predictable/clear), Robust (works across assistive technologies and browsers).

Q17. How do you make a custom dropdown/modal accessible?

Use correct ARIA roles (`role="dialog"`, `aria-modal="true"`), manage focus (trap focus inside modal, return focus to trigger element on close), support `Escape` to close and `Tab`/`Shift+Tab` to cycle, and label with `aria-labelledby`/`aria-label`.

Q18. What's the difference between `aria-label`, `aria-labelledby`, and `aria-describedby`?

`aria-label` provides a direct accessible name as a string. `aria-labelledby` points to another element's ID whose text becomes the accessible name (useful when the label text is already visible elsewhere). `aria-describedby` points to an element providing extra descriptive text (like a hint or error message), read after the label.

Q19. Why should you avoid `<div onClick>` for interactive elements?

A `<div>` isn't keyboard-focusable or announced as interactive by screen readers by default — keyboard/screen reader users can't discover or activate it. Use a real `<button>`, or if a div is unavoidable, add `role="button"`, `tabIndex={0}`, and handle `onKeyDown` for Enter/Space.

Q20. How do you test accessibility in a React app (tools/approach)?

Automated: `jest-axe` in unit tests, Lighthouse/axe DevTools browser extension for manual audits. Manual: navigate the app using only keyboard (Tab/Shift+Tab/Enter/Escape), and test with a screen reader (VoiceOver/NVDA) for critical flows. Automated tools catch ~30-40% of issues — manual keyboard/screen-reader testing is still essential.

Security Basics

Q21. What is XSS (Cross-Site Scripting), and how does React help prevent it?

XSS is injecting malicious scripts into a page, often via unsanitized user input rendered as HTML. React escapes values rendered via JSX by default (`{userInput}` is treated as text, not HTML), which prevents most XSS. The danger zone is `dangerouslySetInnerHTML` — only use it with sanitized content (e.g., via `DOMPurify`).

Q22. What is CSRF, and how is it typically mitigated?

CSRF (Cross-Site Request Forgery) tricks a logged-in user's browser into making an unwanted request to a site they're authenticated on. Mitigations: CSRF tokens tied to the user session, `SameSite=Strict/Lax` cookies, checking the `Origin`/`Referer` header on state-changing requests.

Q23. What is CORS, and why does it exist?

CORS (Cross-Origin Resource Sharing) is a browser security mechanism that blocks a webpage from making requests to a different origin (domain/port/protocol) unless the server explicitly allows it via `Access-Control-Allow-Origin` headers. It exists to prevent malicious sites from silently reading data from other origins using a logged-in user's credentials.

Q24. Where should you store an auth token — localStorage, sessionStorage, or cookies — and why?

`localStorage`/`sessionStorage` are readable by any JS on the page, making them vulnerable to XSS-based token theft. `httpOnly` cookies (inaccessible to JS) are generally safer for auth tokens, combined with `Secure` and `SameSite` flags, though they need CSRF protection since cookies auto-attach to requests.

System Design for Frontend (increasingly common in senior FE rounds)

Q25. How would you design a reusable, scalable Button component for a design system?

Talking points: prop-driven variants (`variant`, `size`, `disabled`) rather than one-off classes, forward `ref` for DOM access, support `as`/polymorphic rendering (render as `<a>` when needed), ensure accessibility (proper `button`/`role`, focus states, `aria-disabled`), use CSS variables/tokens for themability, and write it with TypeScript generics if polymorphic.

Q26. How would you design an infinite-scrolling feed (like a news feed)?

Talking points: `IntersectionObserver` to detect when the user nears the bottom and trigger the next page fetch, paginate via cursor-based API (more stable than offset-based under concurrent inserts), virtualize the list (react-window/react-virtual) so the DOM doesn't grow unbounded, show skeleton loaders, and handle race conditions (cancel stale requests when the user scrolls fast).

Q27. How would you design a client-side caching layer for API responses?

Talking points: cache by request key (URL + params), set TTL/staleness rules, support

cache invalidation on mutation, consider using existing solutions (React Query/TanStack Query, SWR, RTK Query) which handle deduping, background refetching, and stale-while-revalidate out of the box rather than hand-rolling this.

Q28. How would you architect a large React app's folder/component structure?

Talking points: feature-based folders (group by domain/feature, not by type) over type-based folders (all components/, all hooks/) for scalability; colocate tests/styles with components; separate "dumb" presentational components from "smart" container/data-fetching components; shared UI kit in a common folder; clear boundaries between features to avoid tight coupling.

Build Tools

Q29. What does a bundler (Webpack/Vite) actually do?

It takes your entry file(s), traces the dependency graph (imports), transforms code (via loaders/plugins — JSX, TS, CSS), and outputs optimized bundles — with tree-shaking (removing unused exports), minification, and code-splitting.

Q30. Why is Vite generally faster in development than Webpack?

Vite serves source files over native ES modules directly to the browser during dev (no full bundle needed), using esbuild (written in Go) for fast pre-bundling of dependencies — so dev server startup and HMR (hot module replacement) are near-instant regardless of app size. Webpack bundles the entire app before serving, which scales worse as the app grows.

Q31. What is tree-shaking, and what's required for it to work?

Tree-shaking removes unused exports from the final bundle. It requires ES Modules (static `import/export`, not CommonJS `require`) because the bundler needs to statically analyze what's actually used at build time.

More DSA (with 5 hrs/day you have room for these too)

Q32. Merge two sorted arrays.

Two-pointer approach, compare heads, push smaller, $O(n+m)$.

Q33. Find the first non-repeating character in a string.

Hashmap of `char` $\rightarrow$ `count` in one pass, then a second pass to find first with count 1. $O(n)$.

Q34. Implement a basic LRU cache.

Use a `Map` (preserves insertion order) — on `get`, delete and re-insert the key to mark it recently used; on `put`, if at capacity, delete the first (oldest) key via `map.keys().next().value`.

Q35. Detect a cycle in a linked list (conceptual — common as a "explain, don't necessarily code" FE question).

Floyd's cycle detection ("tortoise and hare") — two pointers moving at different speeds; if they meet, there's a cycle.

Q36. Implement `Array.prototype.flat(depth)` from scratch.

```
js
function myFlat(arr, depth = 1) {
  return depth > 0
    ? arr.reduce((acc, val) => acc.concat(Array.isArray(val) ? myFlat(val, depth) : arr.slice()), []);
}
```

Q37. Find the longest common prefix among an array of strings.

Sort the array, compare only the first and last strings character by character (the extremes bound the answer) — $O(n \log n)$.

Q38. Implement a basic event emitter (common FE system-design-lite question).

```
js
class EventEmitter {
  constructor() { this.events = {}; }
  on(event, cb) { (this.events[event] || []).push(cb); }
  emit(event, ...args) { (this.events[event] || []).forEach((cb) => cb(...args)); }
  off(event, cb) { this.events[event] = (this.events[event] || []).filter((fn) => fn !== cb); }
}
```

More TypeScript

Q39. What are mapped types? Give an example.

Mapped types build a new type by transforming each property of an existing type.

ts

```
type Readonly<T> = { readonly [K in keyof T]: T[K] };  
type Optional<T> = { [K in keyof T]?: T[K] };
```

Q40. What is a conditional type?

A type that resolves based on a condition, using `extends ? :`.

ts

```
type IsString<T> = T extends string ? true : false;  
type A = IsString<'hi'>;    // true  
type B = IsString<42>;     // false
```

Q41. What does `keyof` do?

Produces a union of a type's property names as string literals.

ts

```
interface User { id: string; name: string; }  
type UserKeys = keyof User; // 'id' | 'name'
```

Q42. How do you type a function that takes a callback with specific arguments?

ts

```
function onFetch(callback: (data: User, error: Error | null) => void) { /* ...
```

Q43. What's the difference between a type assertion (`as`) and type casting in other languages?

TypeScript's `as` doesn't perform any runtime conversion — it only tells the compiler "trust me, treat this as type X," purely a compile-time construct. It doesn't validate or transform the actual value, so an incorrect assertion can still cause runtime errors.

Quick Additions to Your Mock-Interview Toolkit

With extra time, actually build these end-to-end once (don't just read them) — they show up very often as live-coding tasks in senior FE interviews:

1. A typed custom `useDebounce` hook.

2. A simple `EventEmitter` class (Q38 above).
 3. A basic `Promise` polyfill (states + `.then` chaining).
 4. An accessible custom `Modal` component with focus trap.
 5. A `useLocalStorage<T>` generic hook (Part 6, Q47 in the main doc).
-

Pair this with `FE_Interview_Prep_Plan.md` (core plan) and `FE_Interview_Flashcards.md` (rapid review) for full coverage.